



AWS

AWS FULL OBSERVABILITY PLATFORM

Centralized Monitoring • Real-Time Alerting • Centralized Logging • Log Analytics

Prepared by: Adhithyan Sivaraman T

Region: ap-south-1 (Mumbai) | AWS Cloud Project

EC2

Lambda

API Gateway

CloudWatch

SNS

S3

Athena

AGENDA

01

Project Overview

Goals, objectives & technology stack

02

Architecture

End-to-end observability architecture diagram

03

Monitoring Setup

SNS, EC2, Lambda & API monitoring

04

Alerting Pipeline

CloudWatch Alarms → SNS → Email alerts

05

Dashboard & Visualization

Unified CloudWatch observability dashboard

06

Centralized Logging

Lambda logs → CloudWatch → S3

07

Log Analytics

Athena queries & insights from logs

08

Results & Future Scope

Outcomes, challenges & enhancements

ARCHITECTURE OVERVIEW

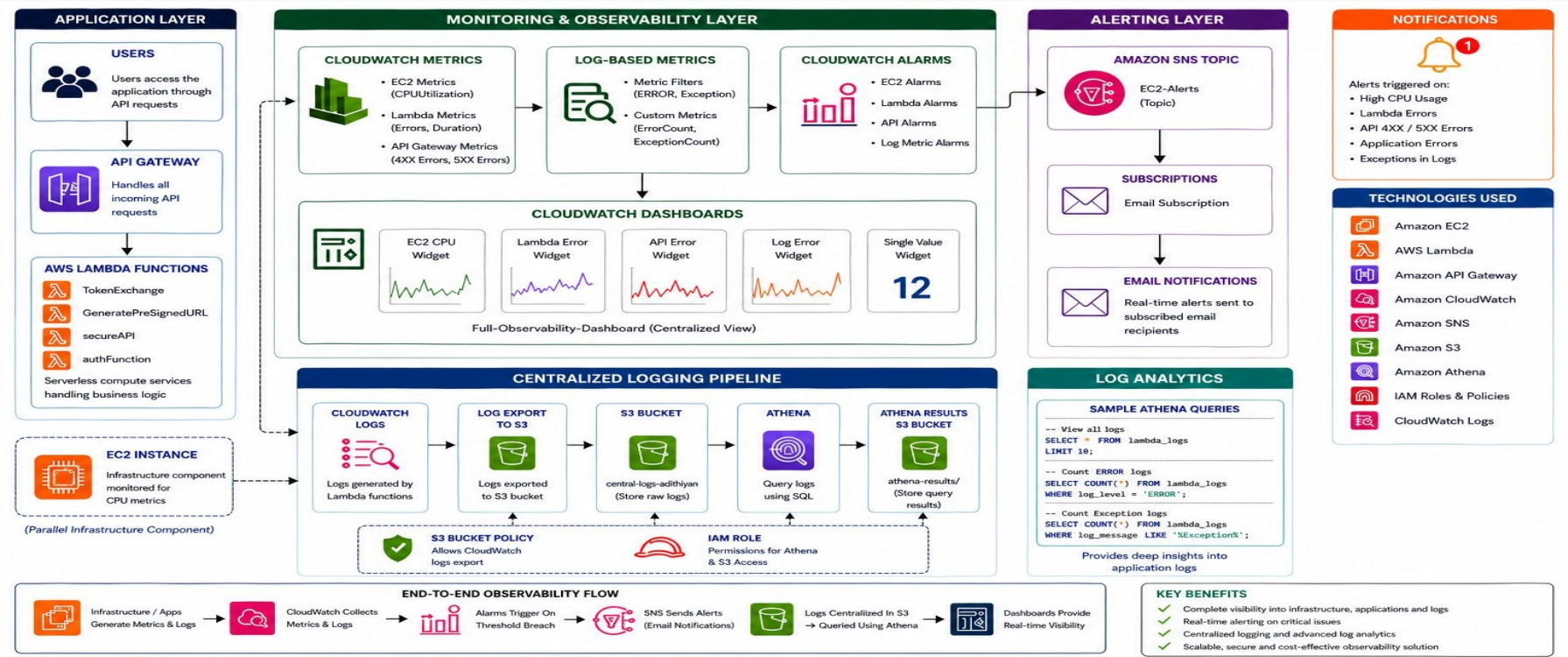
ap-south-1 Mumbai

AWS-Full-Observability-Architecture.png

AWS FULL OBSERVABILITY PLATFORM – ARCHITECTURE

Centralized Monitoring • Centralized Logging • Real-time Alerts • Dashboards • Log Analytics

Region: ap-south-1 (Mumbai)



TECHNOLOGY STACK



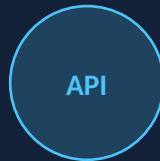
Amazon EC2

Virtual server monitored for CPU utilization metrics via CloudWatch



AWS Lambda

Serverless functions generating logs, errors & invocation metrics



API Gateway

Tracks 4XX/5XX HTTP errors and API request volume



CloudWatch

Central observability — metrics, logs, alarms and dashboards



Amazon SNS

Publishes alarm notifications to email subscribers in real-time



Amazon S3

Centralized long-term storage for exported CloudWatch logs



Amazon Athena

Serverless SQL query engine for log analytics over S3 data

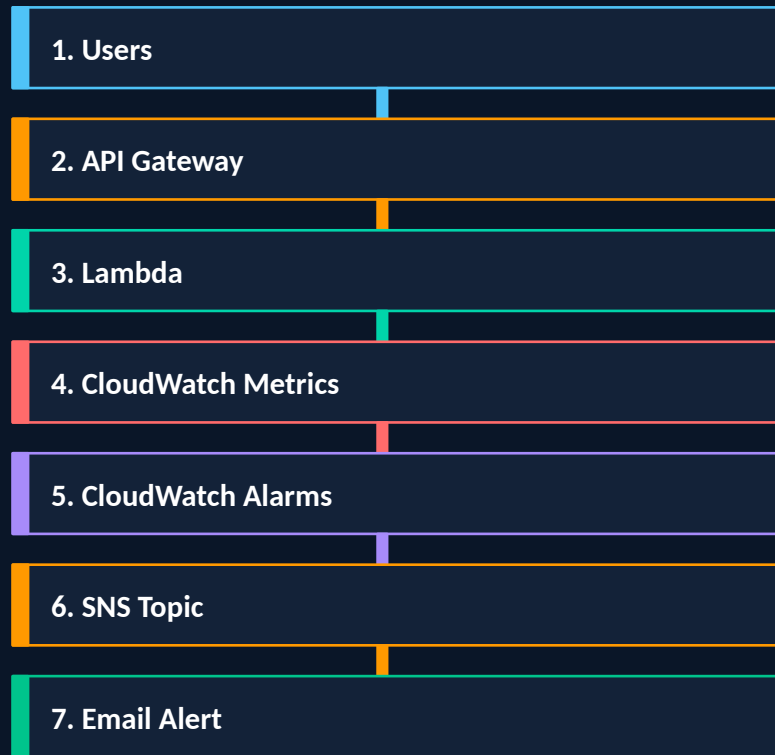


IAM

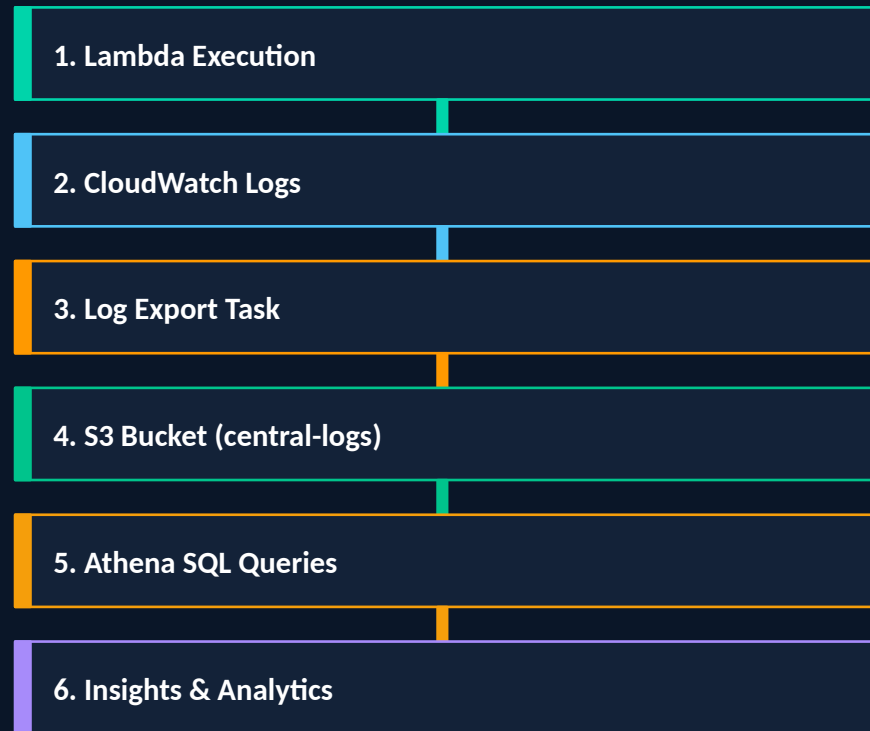
Manages permissions for S3 access, log exports and Athena queries

PROJECT WORKFLOW

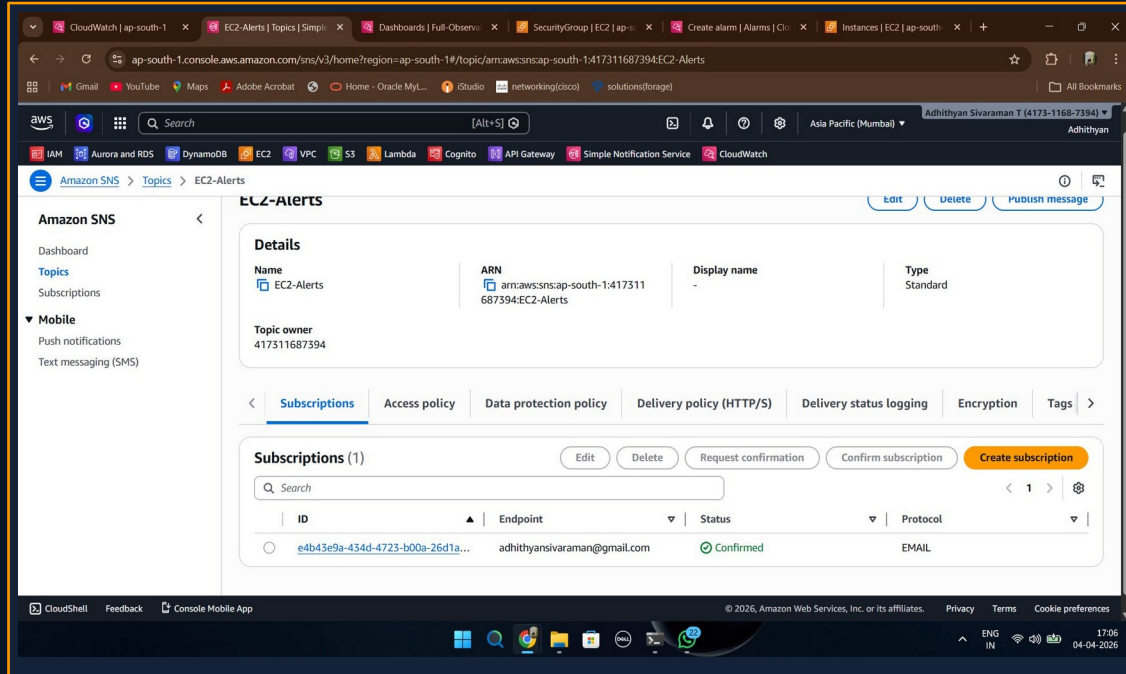
MONITORING & ALERTING PIPELINE



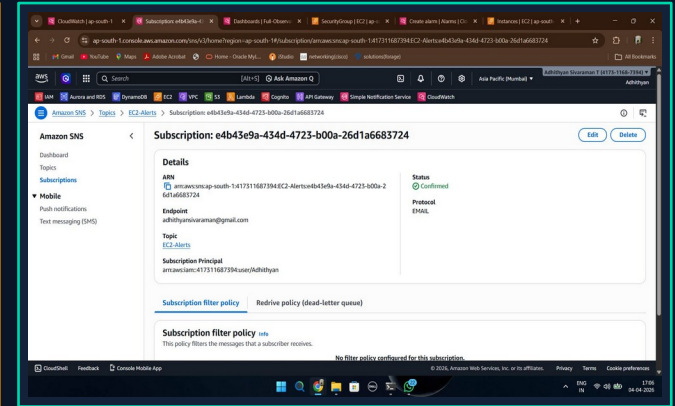
CENTRALIZED LOGGING PIPELINE



SNS TOPIC & EMAIL SUBSCRIPTION



SNS Topic Creation



Email Subscription Confirmation

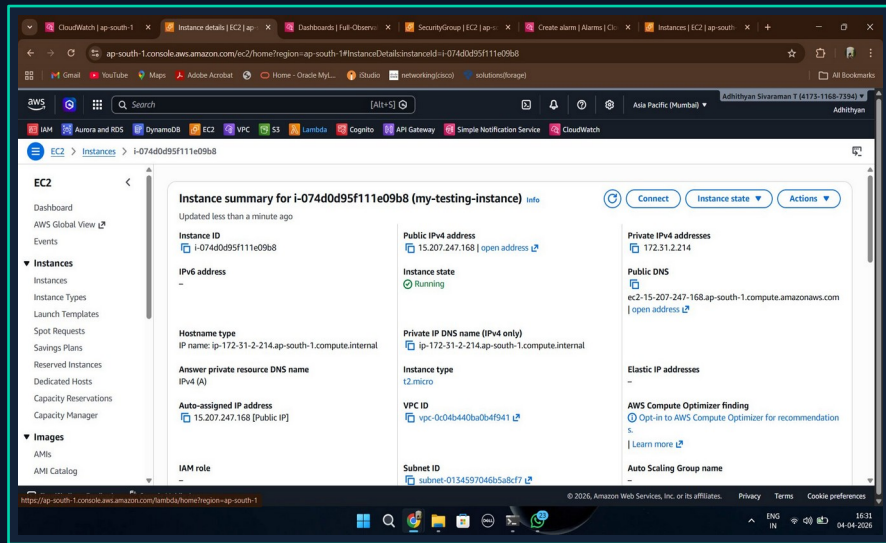
Topic Name: EC2-Alerts (Standard)

Protocol: EMAIL

Subscriber: adhithyansivaraman@gmail.com

Status: ✓ Confirmed

EC2 & LAMBDA MONITORING

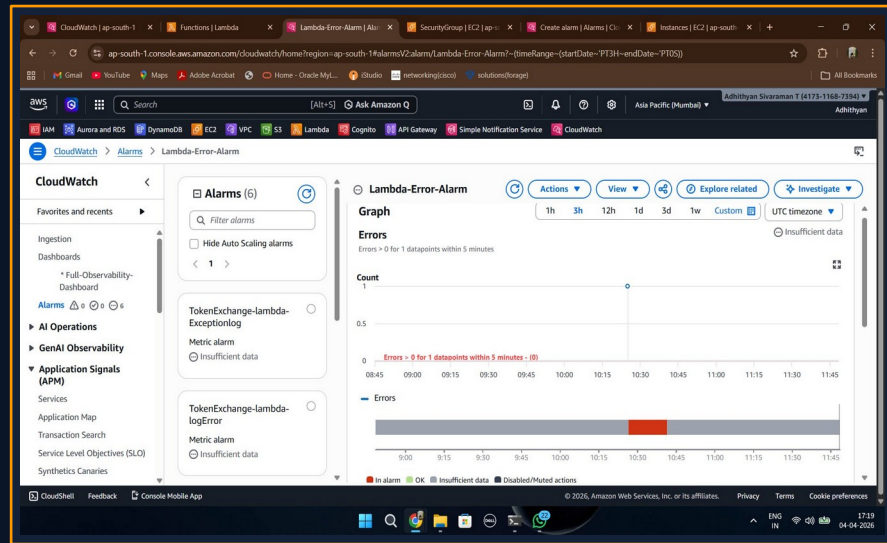


4-EC2-Instance-Running.png

Instance: i-074d0d95f11e09b8 | Type: t2.micro | Status: Running | Metric: CPUUtilization

EC2 INFRASTRUCTURE MONITORING

Purpose: Monitor infrastructure and serverless application health in real time.

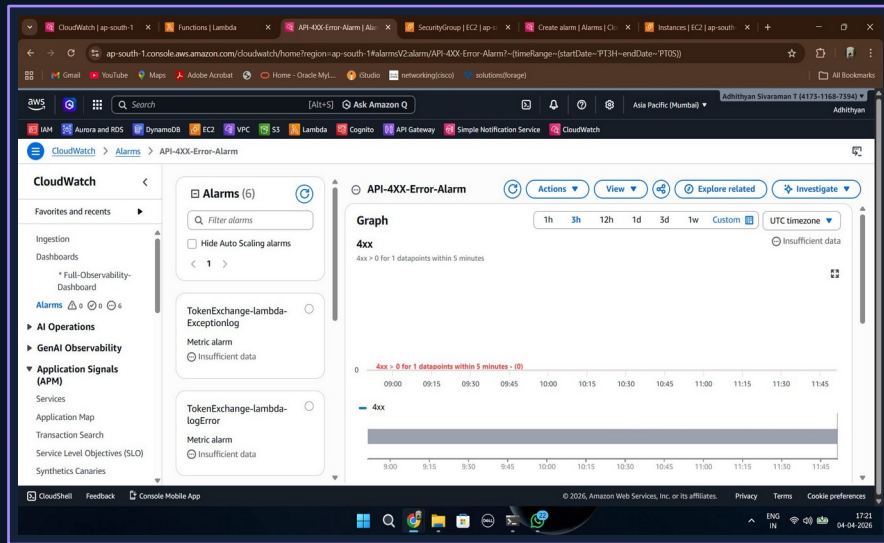


8-Lambda-Error-Alarm-Setup.png

Alarm: Lambda-Error-Alarm | Metric: Errors > 0 | Period: 5 min | Action: Notify SNS

LAMBDA ERROR ALARM SETUP

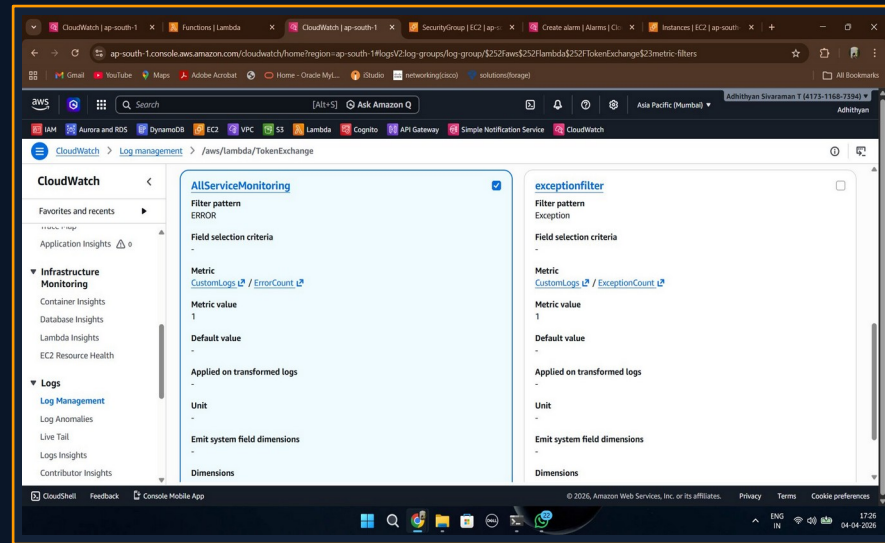
API MONITORING & METRIC FILTERS



10-API-Gateway-Metrics-Selection.png

4XX Errors — client-side request failures tracked

5XX Errors — server-side faults monitored in real-time

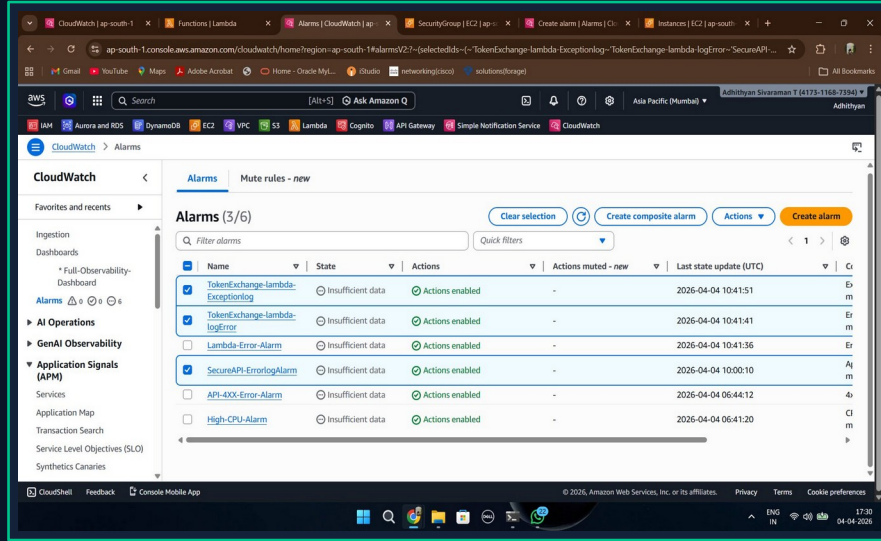


13-Metric-Filter-ERROR.png

AllServiceMonitoring: Filter pattern = ERROR → ErrorCount

exceptionfilter: Filter pattern = Exception → ExceptionCount

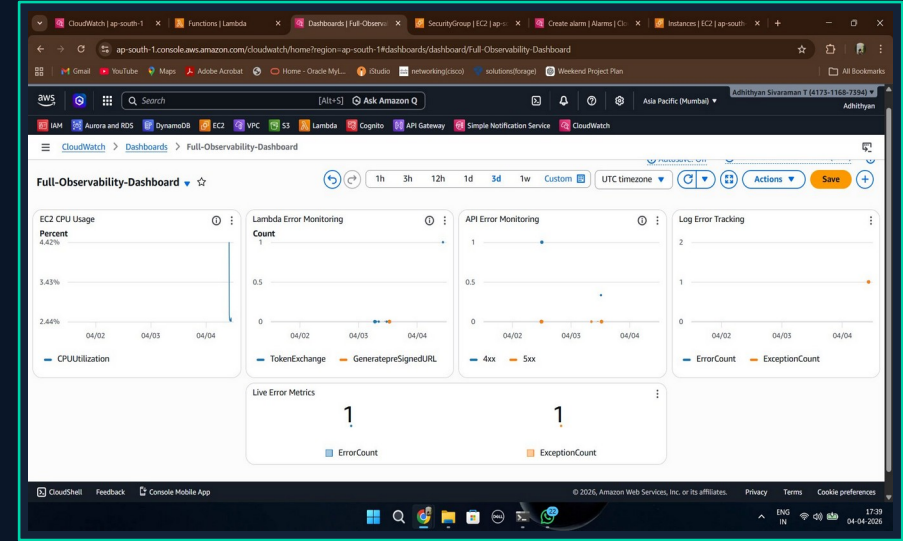
CLOUDWATCH ALARMS & DASHBOARD



16-Log-Alarms-Created.png

6 ALARMS CONFIGURED:

- TokenExchange-lambda-Exceptionlog
- TokenExchange-lambda-logError
- Lambda-Error-Alarm
- SecureAPI-ErrorlogAlarm
- API-4XX-Error-Alarm
- High-CPU-Alarm

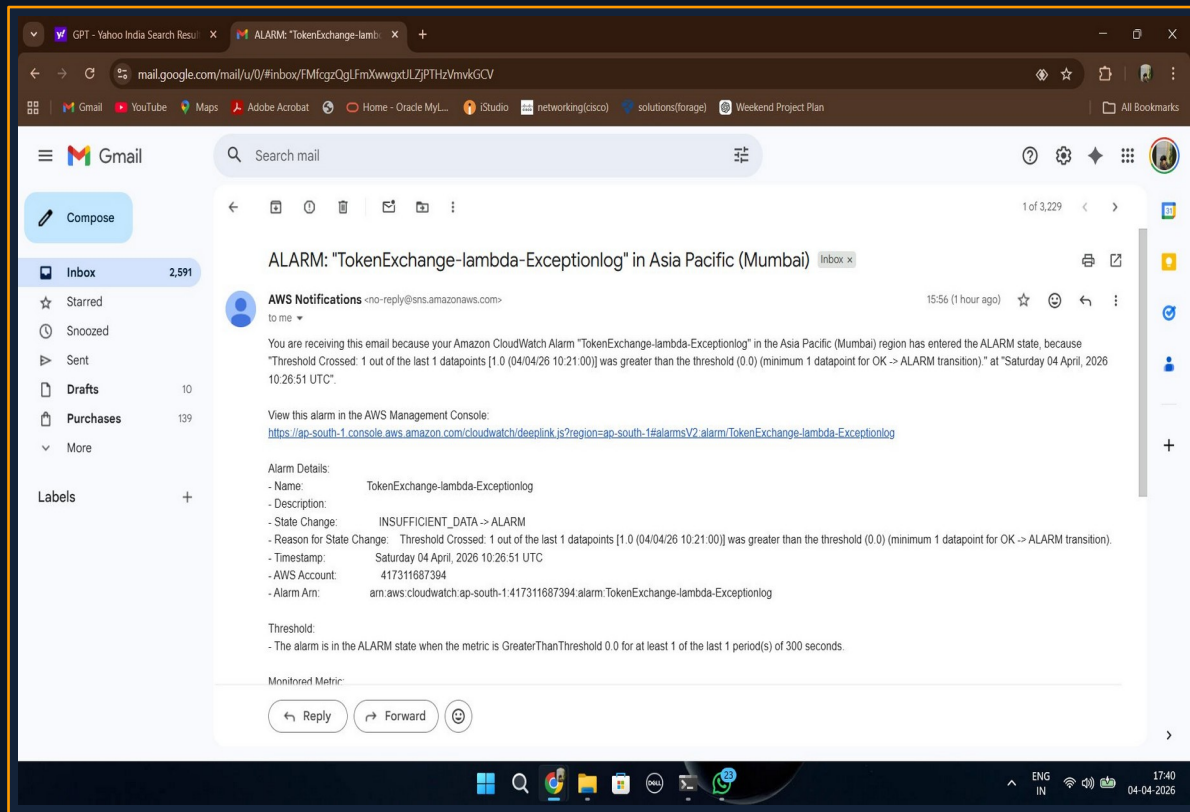


23-FINAL-DASHBOARD-VIEW.png

DASHBOARD WIDGETS:

- ✓ EC2 CPU Usage
- ✓ Lambda Error
- ✓ API Error
- ✓ Log Error Tracking
- ✓ Live Metrics (Single)

REAL-TIME EMAIL ALERT — END-TO-END VALIDATION



24-email-Alert-recieved.png

ALERT DETAILS

Alarm Name

TokenExchange-lambda-Exceptionlog

State Change

INSUFFICIENT_DATA → ALARM

Threshold

ExceptionCount ≥ 1 in 5 min

Timestamp

Saturday, 04 April 2026, 10:26:51 UTC

AWS Account

417311687394

Region

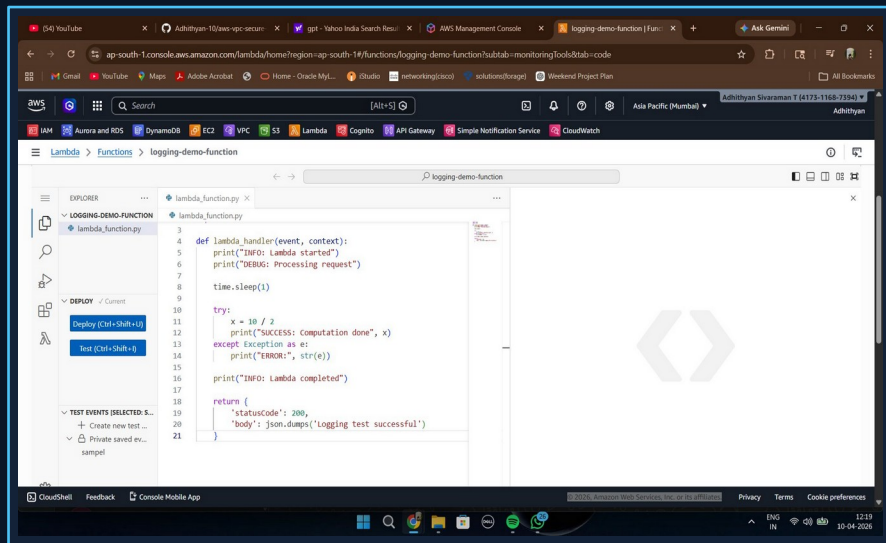
Asia Pacific (Mumbai)

Delivery

SNS → Email (confirmed subscription)

Validation Outcome: End-to-end alert flow tested successfully ✓

CENTRALIZED LOGGING IMPLEMENTATION



1-Lambda-Function-Code.png

INFO

Logs Lambda start and completion events

DEBUG

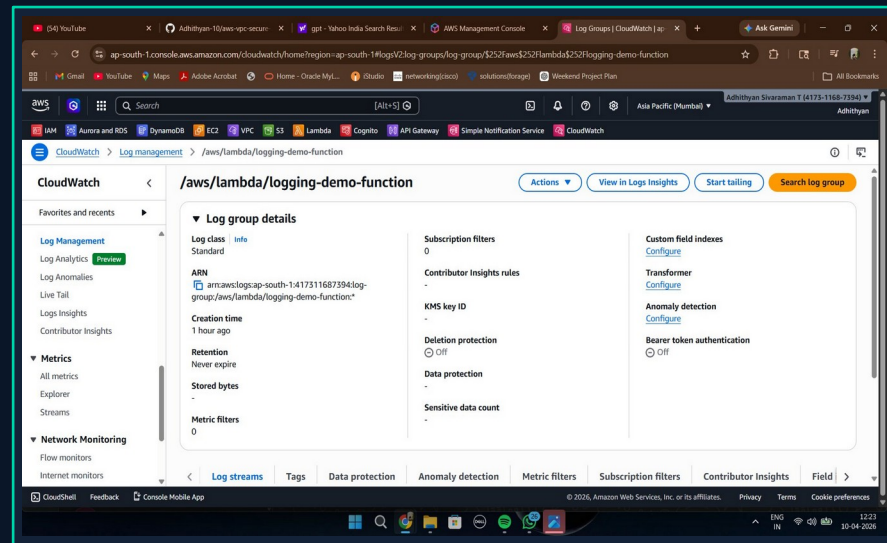
Tracks processing request messages

SUCCESS

Confirms successful computation results

ERROR

Catches and logs exception details



3-CloudWatch-Log-Group.png

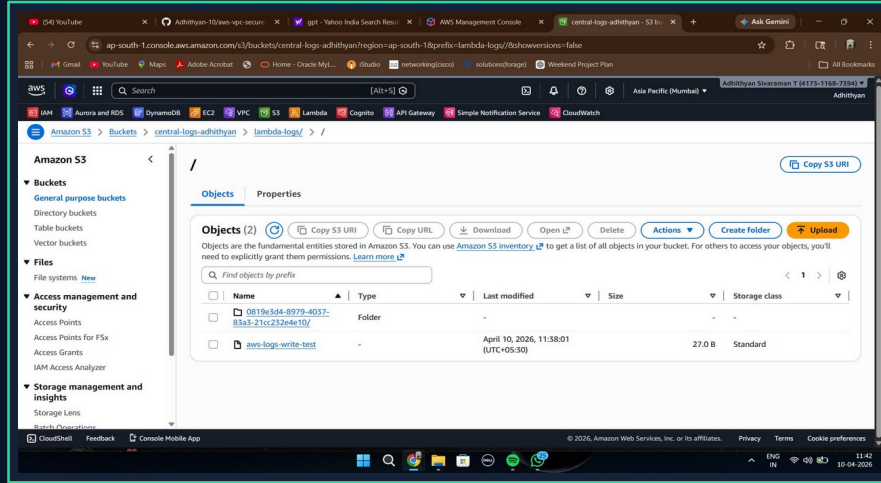
- Log Group: /aws/lambda/logging-demo-function

- Retention: Never expire | Log Class: Standard

- ARN: arn:aws:logs:ap-south-1:417311687394:...

- Log Streams auto-created per Lambda execution

S3 STORAGE & ATHENA LOG ANALYTICS



6-S3-Bucket-(Logs-Stored).png

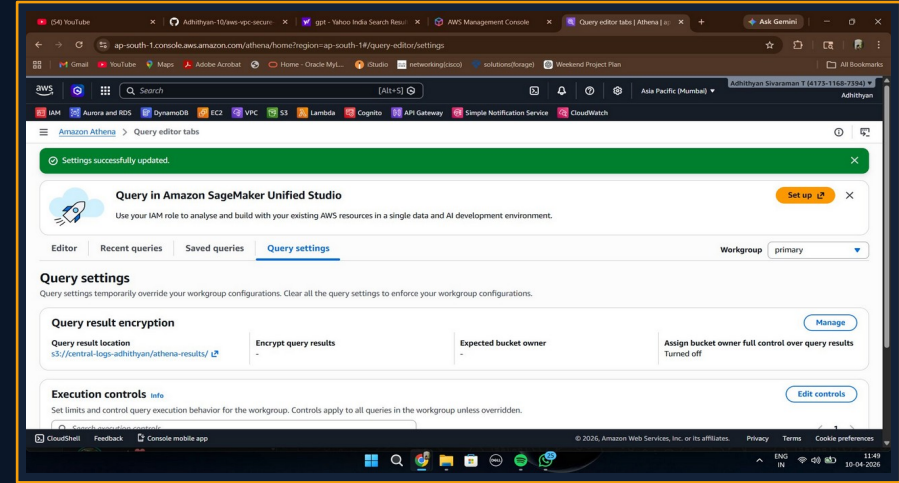
S3 BUCKET DETAILS

Bucket: central-logs-adhithyan

Prefix: lambda-logs/

Objects: 2 (folder + write test)

Storage: Standard class



8-Athena-Query-Settings.png

ATHENA CONFIGURATION

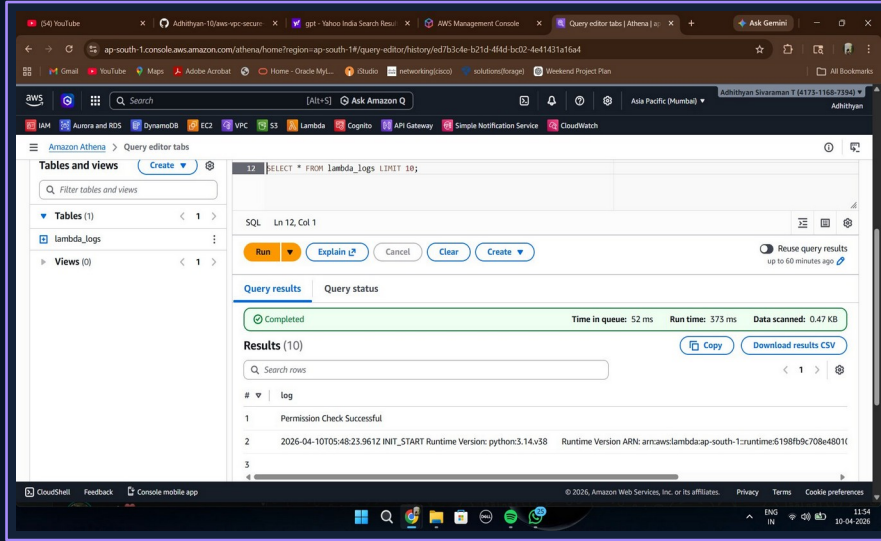
• Query Result Location: s3://central-logs-adhithyan/athena-results/

• Workgroup: primary

• Settings updated successfully

• Encryption: Disabled (test environment)

ATHENA QUERY EXECUTION & RESULTS



10-Athena-Query-Results.png

SELECT * FROM lambda_logs LIMIT 10;

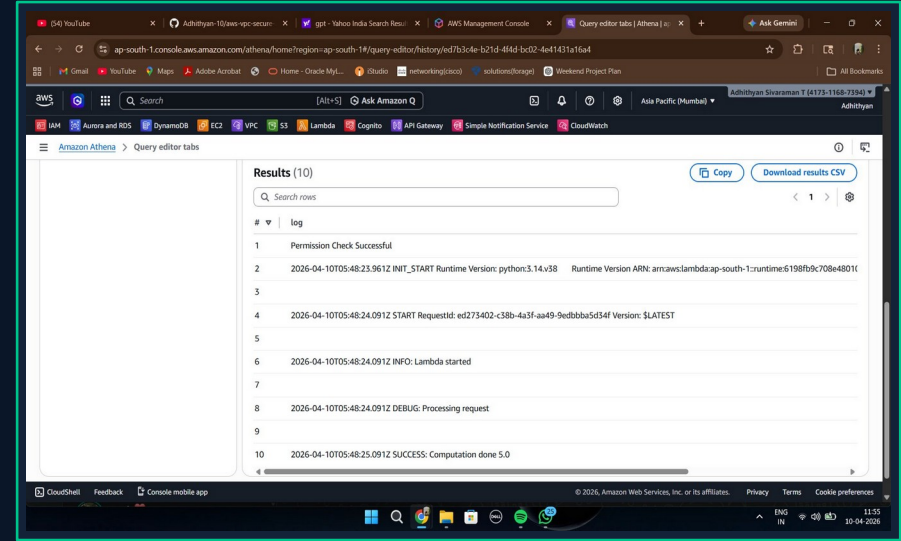
Use Case: Analyze operational logs and troubleshoot failures quickly.

Results: 10 rows returned

Run time: 373 ms | Queue: 52 ms

Data scanned: 0.47 KB

Status: Completed ✓



11-Athena-Result.png

LOG ROWS RETRIEVED:

1. Permission Check Successful
2. INIT_START Runtime: python3.14.v38
3. START RequestId: ed273402...
4. INFO: Lambda started
5. DEBUG: Processing request
6. SUCCESS: Computation done 5.0

SAMPLE ATHENA SQL QUERIES

Query 1 — View All Logs

1

```
SELECT *  
FROM lambda_logs  
LIMIT 10;
```

Retrieves the first 10 log entries from the lambda_logs table stored in S3. Useful for inspecting raw log structure and validating data format.

Query 2 — Count ERROR Logs

2

```
SELECT COUNT(*)  
FROM lambda_logs  
WHERE log_level = 'ERROR';
```

Counts all rows where the log level is ERROR. Provides a quick summary of total error events generated by Lambda functions over time.

Query 3 — Count Exception Logs

3

```
SELECT COUNT(*)  
FROM lambda_logs  
WHERE log_message  
      LIKE '%Exception%';
```

Searches log messages containing the keyword 'Exception'. This helps identify runtime exceptions even when the log level isn't explicitly ERROR.

RESULTS, CHALLENGES & FUTURE SCOPE

KEY RESULTS

- ✓ 6 CloudWatch Alarms configured and active
- ✓ Unified Full-Observability-Dashboard created
- ✓ Email alerts delivered via SNS in real-time
- ✓ Lambda logs exported to central S3 bucket
- ✓ Athena SQL queries return log data in <400ms
- ✓ EC2, Lambda, API & log errors all monitored
- ✓ Centralized visibility reduced troubleshooting complexity

CHALLENGES

- ⚠ IAM permissions for CloudWatch log export to S3
- ⚠ Athena table schema alignment with log format
- ⚠ SNS subscription confirmation email handling
- ⚠ Log metric filter pattern matching & tuning
- ⚠ Alarm testing with insufficient data state
- ⚠ S3 bucket policy for CloudWatch access

FUTURE ENHANCEMENTS

AWS X-Ray

Distributed tracing

OpenSearch

Full-text log search

Slack Alerts

Team notifications

AI Anomaly

Auto-detect spikes

Multi-Region

Global observability

Containers

ECS/EKS monitoring

This project successfully demonstrates a production-grade AWS observability platform with centralized monitoring, real-time alerting, log aggregation, and SQL-based analytics — all deployed in ap-south-1 (Mumbai).

CONCLUSION

AWS Full Observability Platform — Successfully Implemented



Infrastructure Monitoring



API Health Monitoring



Lambda Error Monitoring



Real-Time Email Alerting



Unified CW Dashboard



Centralized Log Storage



Athena Log Analytics



End-to-End Observability